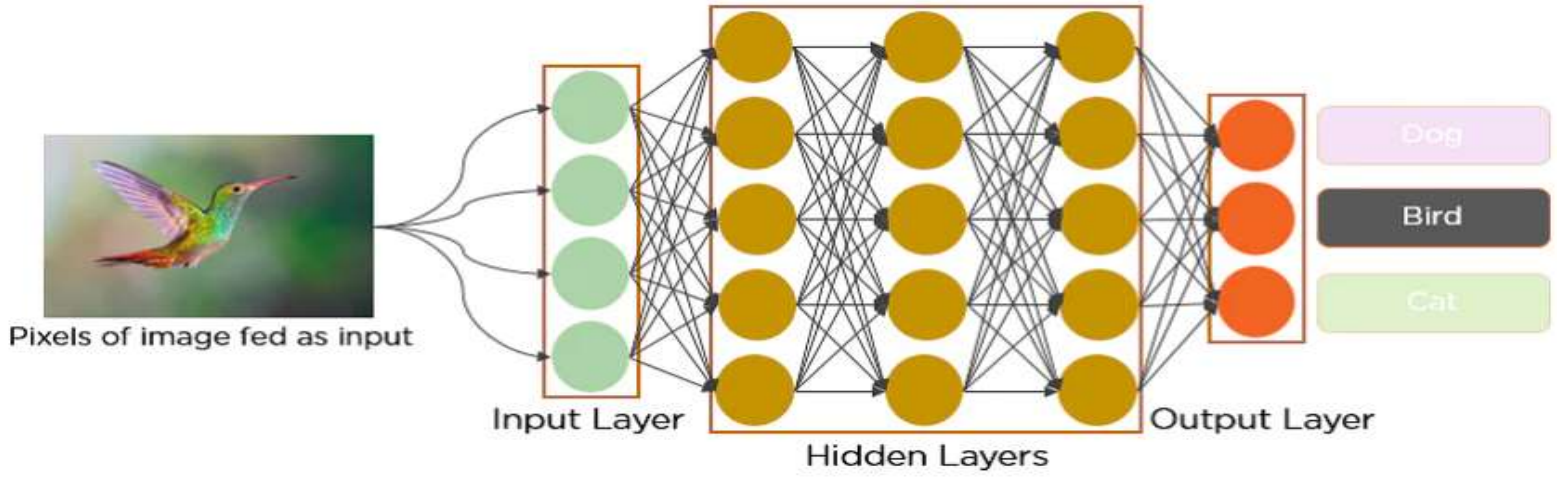
Convolutional Neural Network (CNN)

H.M. Samadhi Chathuranga Rathnayake

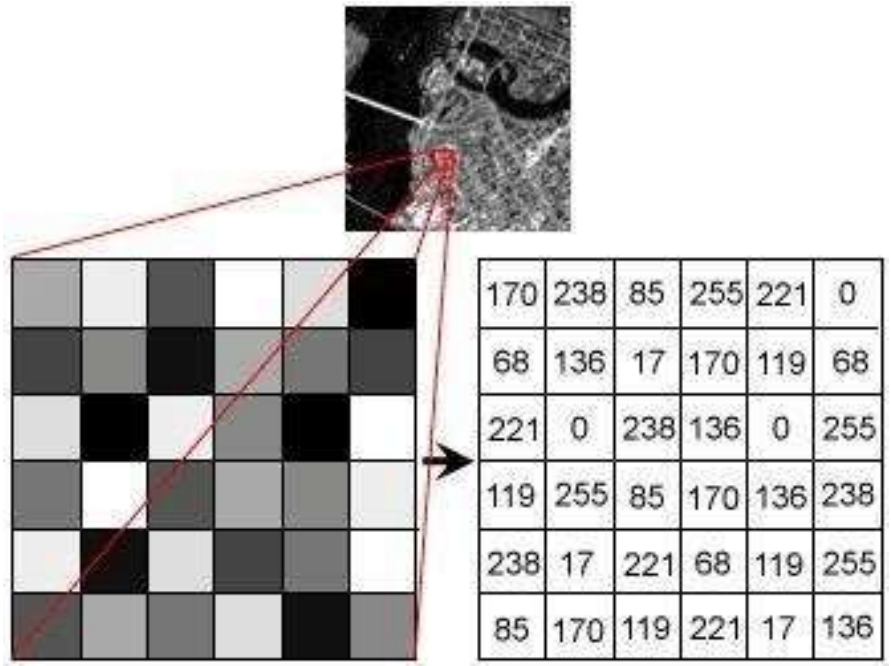
Introduction to CNN

- Convolutional Neural Networks (CNNs) are a type of deep learning algorithm specifically designed for working with grid-like data structures, such as images.
- They have revolutionized computer vision by achieving high accuracy in tasks like image classification, object detection, and segmentation.
- It has two parts; Feature Extraction and Classification.



Representation of Pixels

- An image is represented as a grid of pixels, where each pixel corresponds to a small portion of the image.
- The values in the pixels represent the intensity or color of that specific part of the image.



170	238	85	255	221	0
68	136	17	170	119	68
221	0	238	136	0	255
119	255	85	170	136	238
238	17	221	68	119	255
85	170	119	221	17	136

Grayscale Image (1 Channel)

- Each pixel in a grayscale image represents an intensity value.
- The intensity value is typically a single number ranging from 0 to 255, where 0 represents black, 255 represents white, and values in between represent varying shades of gray.

Color Image (3 Channels)

- Each pixel in a color image is represented by three values corresponding to the Red, Green, and Blue (RGB) channels.
- Each channel typically has a value ranging from 0 to 255.
- For example, a pixel value of (255, 0, 0) represents red, (0, 255, 0) represents green, (0, 0, 255) represents blue, and (255, 255, 255) represents white.

Pixel Representation of Grey Scale Images

- For a grayscale image, consider the following small 3x3 pixel image

$$\begin{bmatrix} 100 & 150 & 200 \\ 50 & 120 & 180 \\ 0 & 60 & 255 \end{bmatrix}$$

Pixel Representation of Color Images

- For a color image, consider the following small 2x2 pixel image with RGB values

$$\begin{bmatrix} (255 & 0 & 0) & (0 & 255 & 0) \\ (0 & 0 & 255) & (255 & 255 & 0) \end{bmatrix}$$

Normalized Pixel Values

- In some cases, pixel values are normalized to a range between 0 and 1, especially when used as input to neural networks.
- Normalization helps in speeding up the training process and can lead to better performance.
- For example, if the original pixel values range from 0 to 255, you can normalize them by dividing by 255.

Grey Scale Image Normalization

- Consider the previous small 3x3 pixel image

$$\begin{bmatrix} 100/255 & 150/255 & 200/255 \\ 50/255 & 120/255 & 180/255 \\ 0/255 & 60/255 & 255/255 \end{bmatrix} = \begin{bmatrix} 0.39 & 0.59 & 0.78 \\ 0.20 & 0.47 & 0.71 \\ 0 & 0.24 & 1 \end{bmatrix}$$

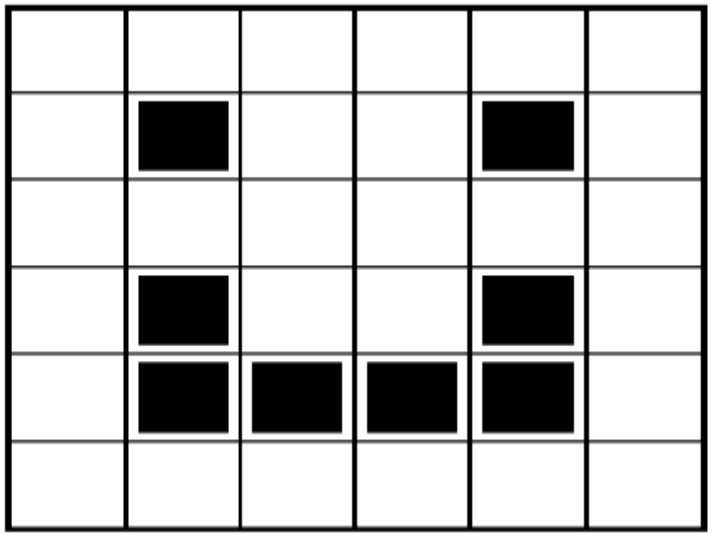
Color Image Normalization

- Consider the previous small 2x2 pixel image with RGB values

$$\begin{bmatrix} (255/255 & 0/255 & 0/255) & (0/255 & 255/255 & 0/255) \\ (0/255 & 0/255 & 255/255) & (255/255 & 255/255 & 0/255) \end{bmatrix} = \begin{bmatrix} (1 & 0 & 0) & (0 & 1 & 0) \\ (0 & 0 & 1) & (1 & 1 & 0) \end{bmatrix}$$

Introduction to CNN

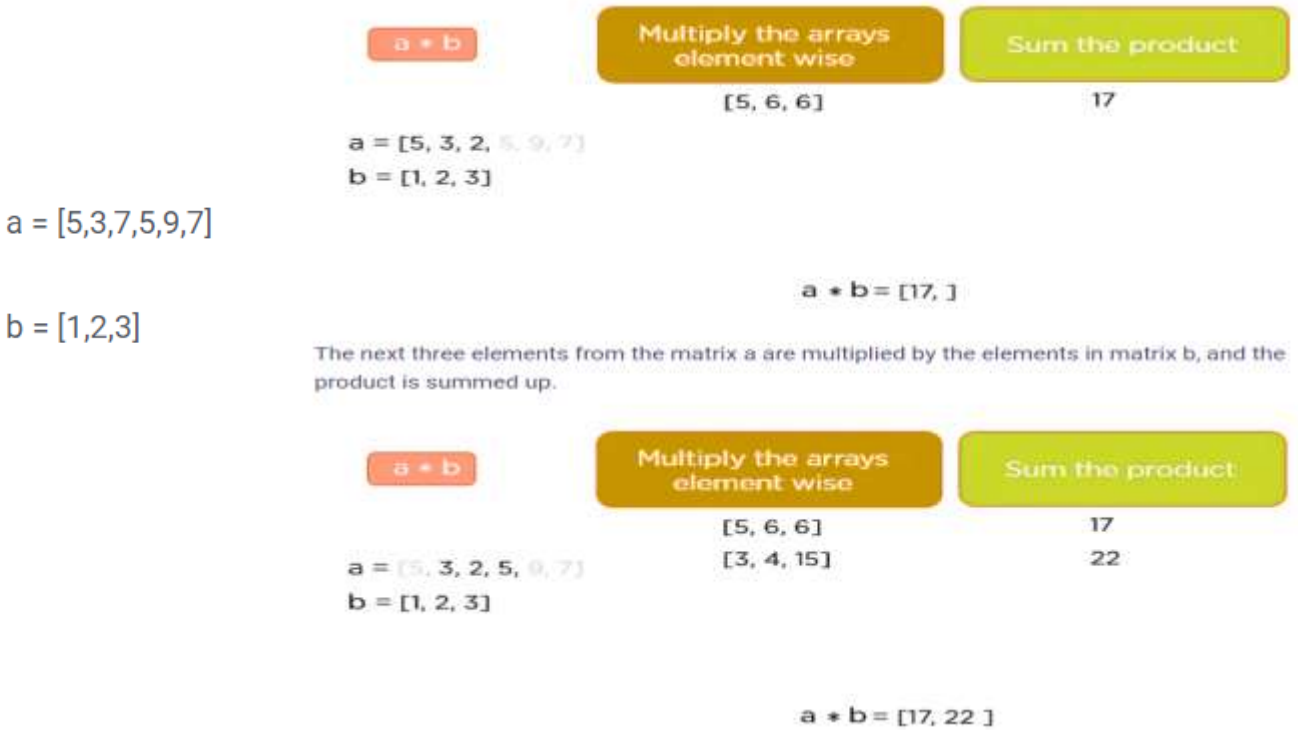
- In CNN, every image is represented in the form of an array of pixel values.



```
1 1 1 1 1 1
1 0 1 1 0 1
1 1 1 1 1 1
1 0 1 1 0 1
1 0 0 0 0 1
1 1 1 1 1 1
```

Convolutional Operations

- Any convolutional neural network's foundation is the convolution process.
- Let's examine the convolution operation using two 1-dimensional matrices, a and b.



- This process continues until the convolution operation is complete.

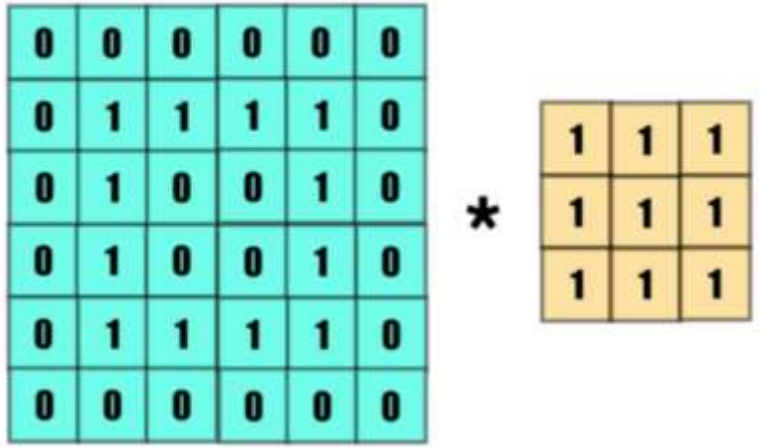
Layers of Convolution Neural Network

- A convolution neural network has multiple hidden layers that help in extracting information from an image. The four important layers in CNN are:

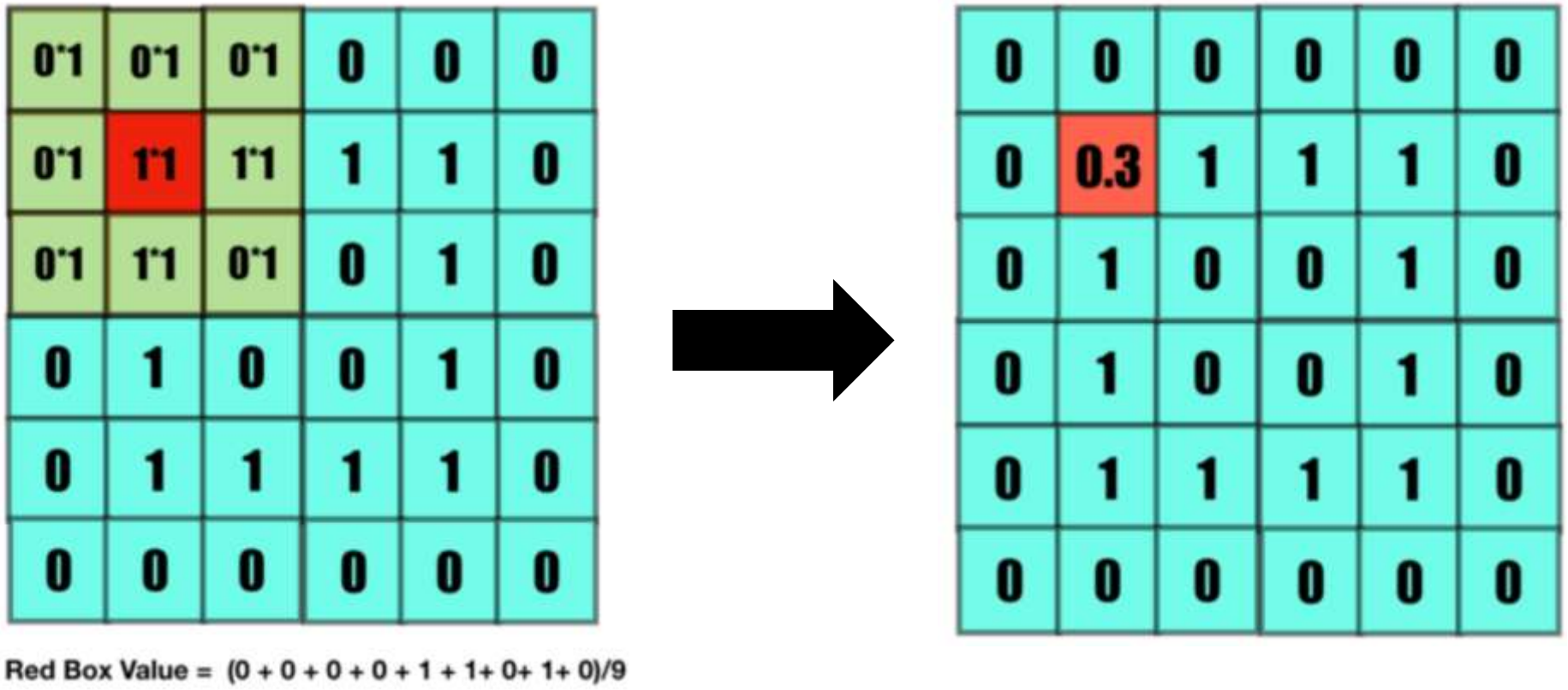
  - Convolution layer
  - ReLU (Rectified Linear Unit) layer
  - Pooling layer
  - Fully connected layer

Convolution Layer

- The process of extracting useful elements from an image begins with this.
- Multiple filters work together to perform the convolution action in a convolution layer.
- Each image can be thought of as a matrix of pixel values.

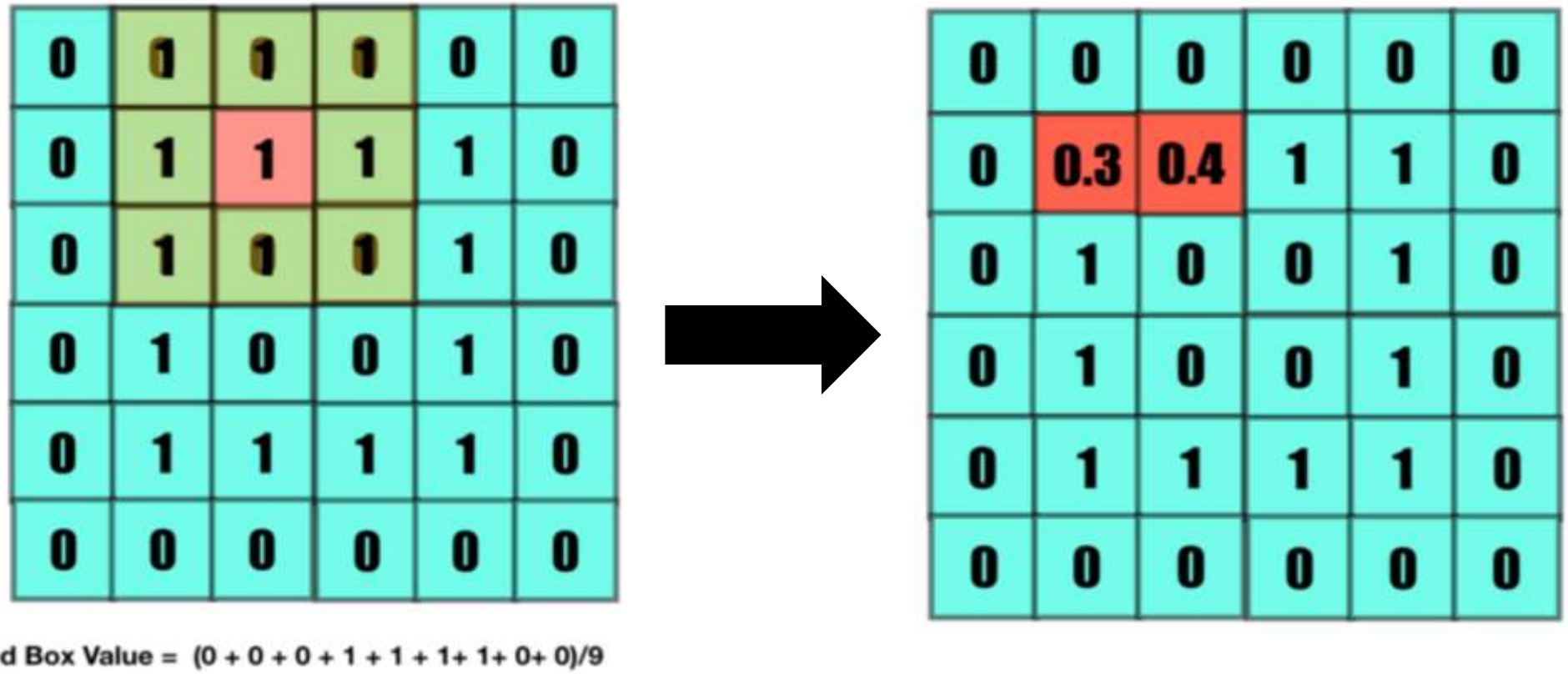


Convolution Layer



Red Box Value = (0 + 0 + 0 + 0 + 1 + 1+ 0+ 1+ 0)/9

Convolution Layer



Red Box Value = (0 + 0 + 0 + 1 + 1 + 1+ 1+ 0+ 0)/9

Convolution Layer

- Get up to

0	0	0	0	0	0
0	0.3	0.4	0.4	0.3	0
0	0.4	0.7	0.7	0.4	0
0	0.4	0.7	0.7	0.4	0
0	0.3	0.4	0.4	0.3	0
0	0	0	0	0	0

Convolution Layer - Stride

- This refers to the number of pixels by which the convolution filter (or kernel) moves across the input image or feature map.
- Stride determines how much the filter shifts at each step as it slides over the input during the convolution operation.
- Ex-
  - A stride of 1 means the filter moves one pixel at a time, which results in the output feature map having a similar size to the input (assuming no padding).
  - A stride of 2 means the filter moves two pixels at a time, which effectively reduces the spatial dimensions of the output feature map.
- The stride affects the size of the output feature map.
- Larger strides lead to smaller output feature maps because the filter overlaps less with the input.

Convolution Layer - Stride

Convolution Operation with Stride 1

INPUT IMAGE

18	54	51	239	244	188
55	121	75	78	95	88
35	24	204	113	109	221
3	154	104	235	25	130
15	253	225	159	78	233
68	85	180	214	245	0

WEIGHT

1	0	1
0	1	0
1	0	1

429

Convolution Operation with Stride 2

INPUT IMAGE

18	54	51	239	244
55	121	75	78	95
35	24	204	113	109
3	154	104	235	25
15	253	225	159	78

WEIGHT

1	0	1
0	1	0
1	0	1

429

Convolution Layer - Stride

- For an input dimension W and a stride S, the output dimension $W*$ is approximately,

$$W^* = \frac{W - Filter\ Size}{S} + 1$$

  - For an input image of size 28x28 with a 3x3 filter and a stride of 1, the output feature map will be 26x26 (assuming no padding).
  - For the same input and filter size but with a stride of 2, the output feature map will be 13x13.
- Higher stride values result in fewer sliding operations, thus reducing the computational load and memory usage.
- However, too high a stride can lead to a loss of detailed information due to the larger steps taken by the filter.

Convolution Layer - Padding

- **Padding** virtually extends the matrix to cater to border values as described in the image below. The pink layer isn't a part of the feature matrix, but helps in convolution.

0	0	0	0	0	0	0	0
0	18	54	51	239	244	188	0
0	55	121	75	78	95	88	0
0	35	24	204	113	109	221	0
0	3	154	104	235	25	130	0
0	15	253	225	159	78	233	0
0	68	85	180	214	245	0	0
0	0	0	0	0	0	0	0

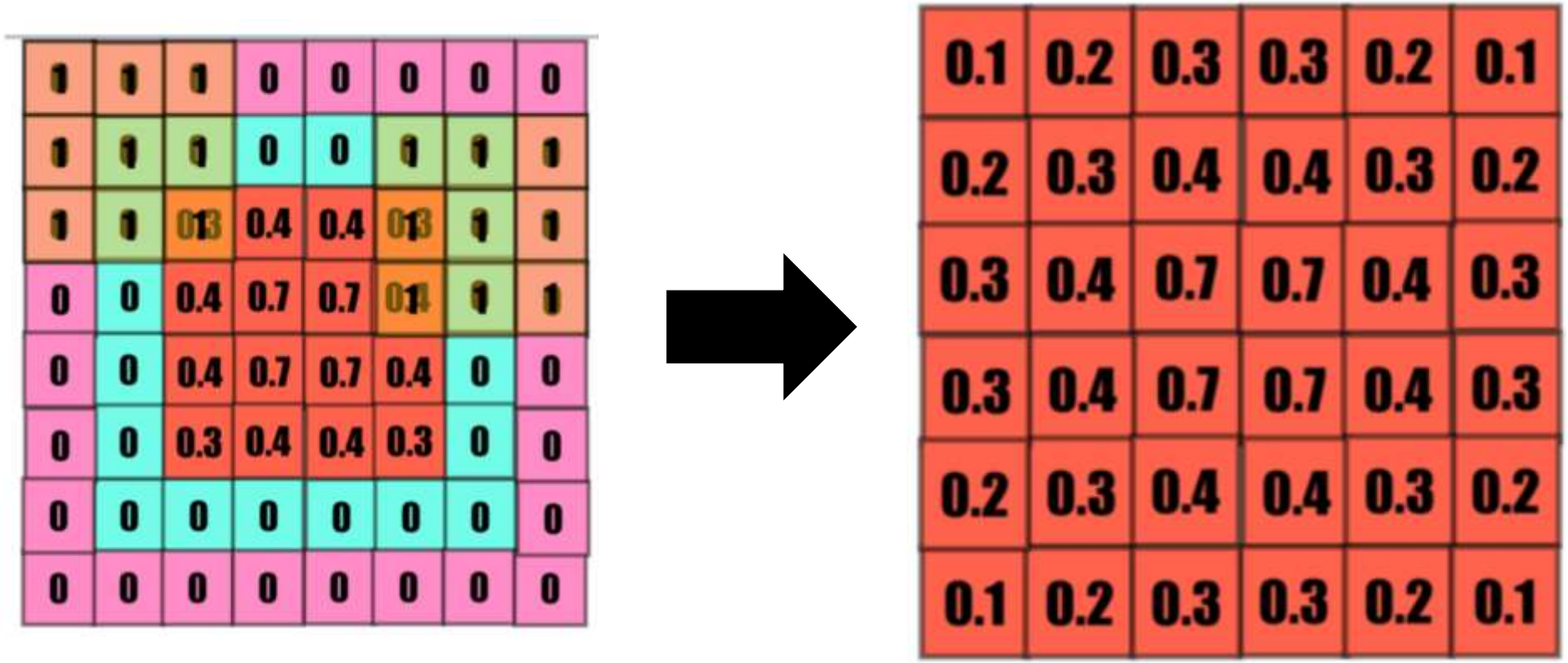
WEIGHT

1	0	1
0	1	0
1	0	1

139

Convolution Layer - Padding

- Get back to the same example.



Convolution Layer

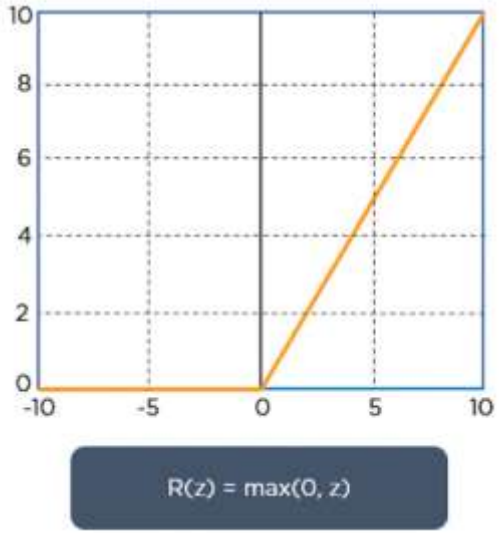
- This is extracting particular information from the original image matrix.

Original Image

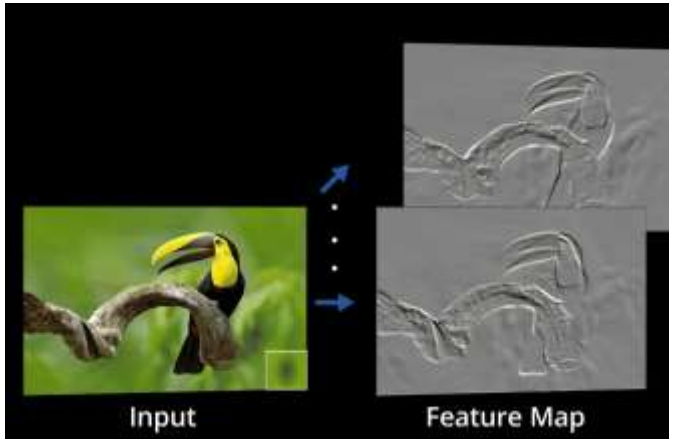
Convoluted image

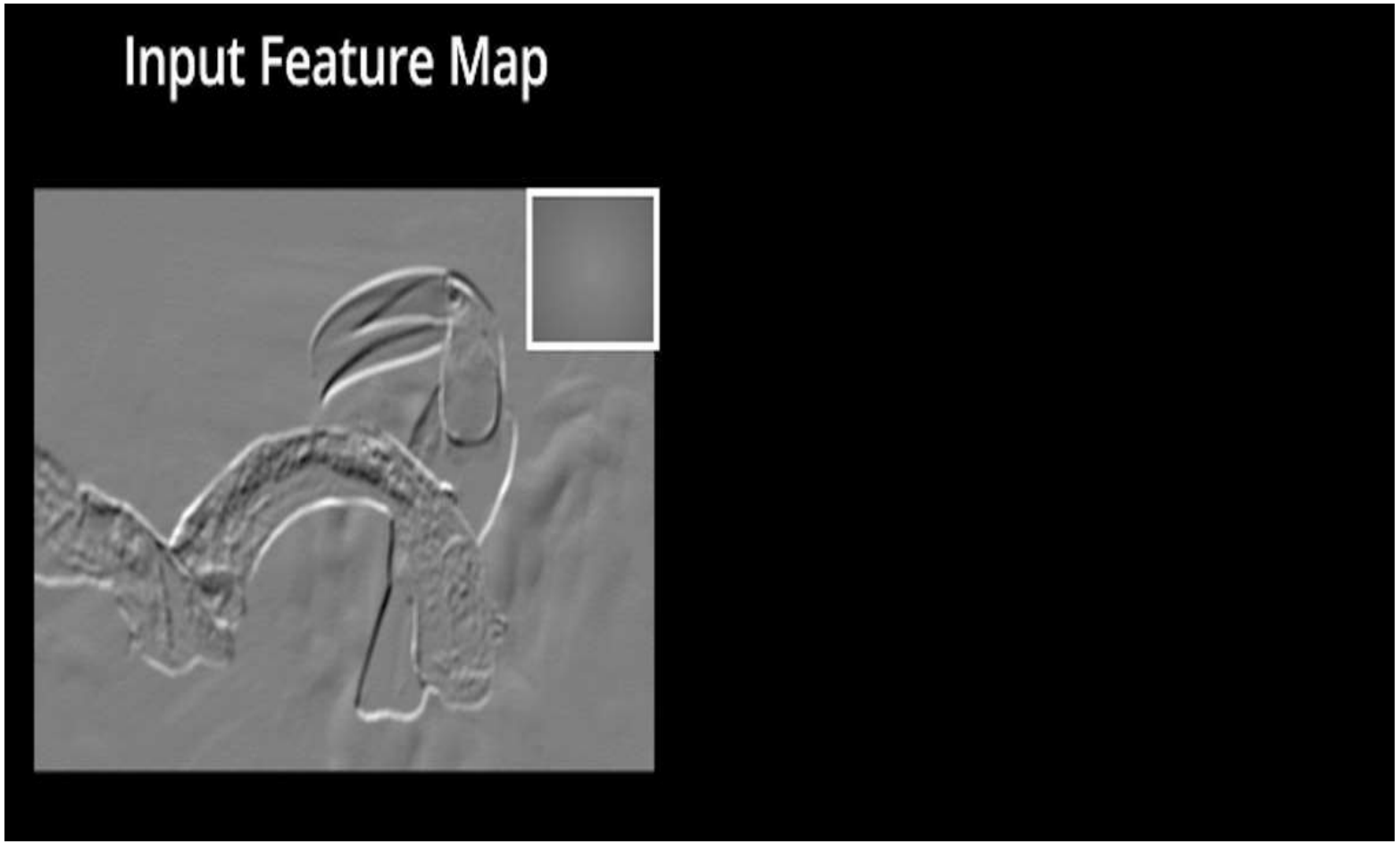
ReLU layer

- ReLU stands for the rectified linear unit.
- Once the feature maps are extracted, the next step is to move them to a ReLU layer.
- ReLU performs an element-wise operation and sets all the negative pixels to 0.
- It introduces non-linearity to the network, and the generated output is a rectified feature map.



ReLU layer



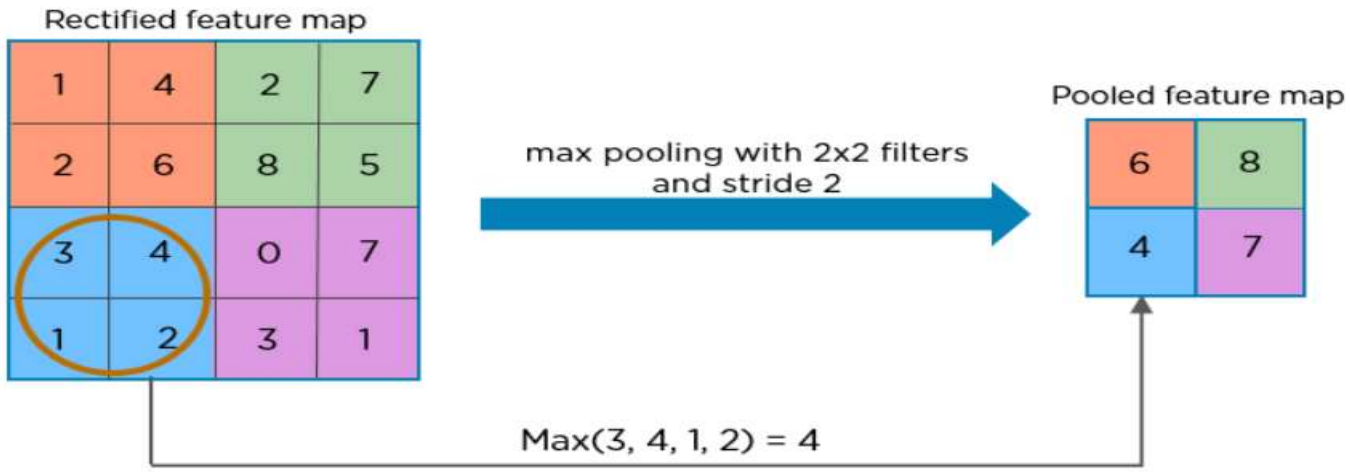


Pooling Layer

- Pooling is a down-sampling operation that reduces the dimensionality of the feature map.
  - Max pooling
  - Average pooling
  - L2 norm pooling
- Pooling is a crucial operation in Convolutional Neural Networks (CNNs) that helps to progressively reduce the spatial dimensions (width and height) of the input representations, reducing the number of parameters and computations in the network.
- It also helps to control overfitting by making the detection of features more robust.

Pooling Layer

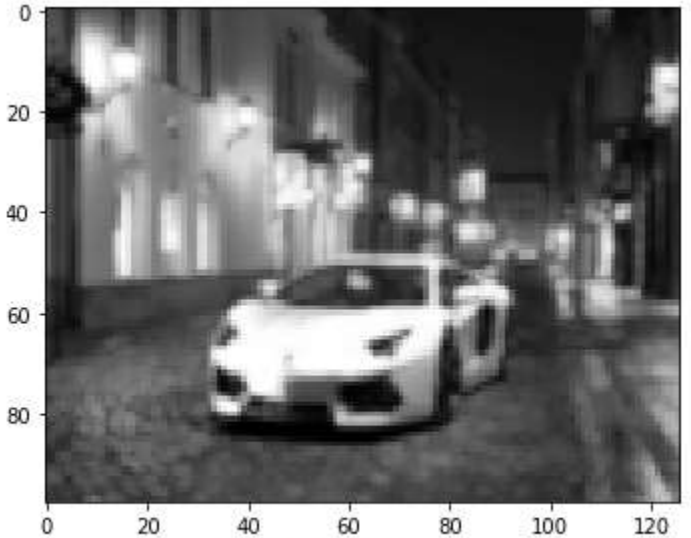
- The rectified feature map now goes through a pooling layer to generate a pooled feature map.



Pooling Layer

- To distinguish distinct portions of the image, such as edges, corners, bodies, feathers, eyes, and beak, the pooling layer employs a variety of filters.

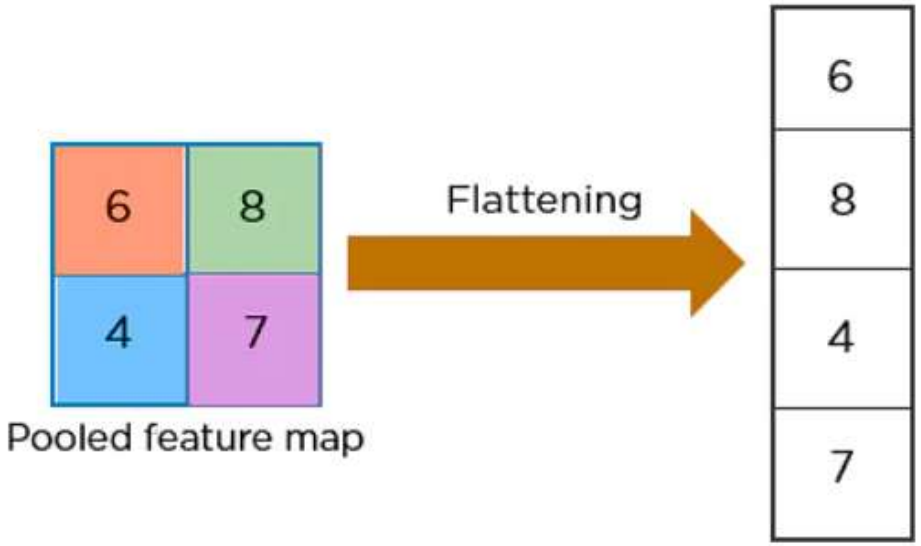
Convoluted image



MAX Pooling

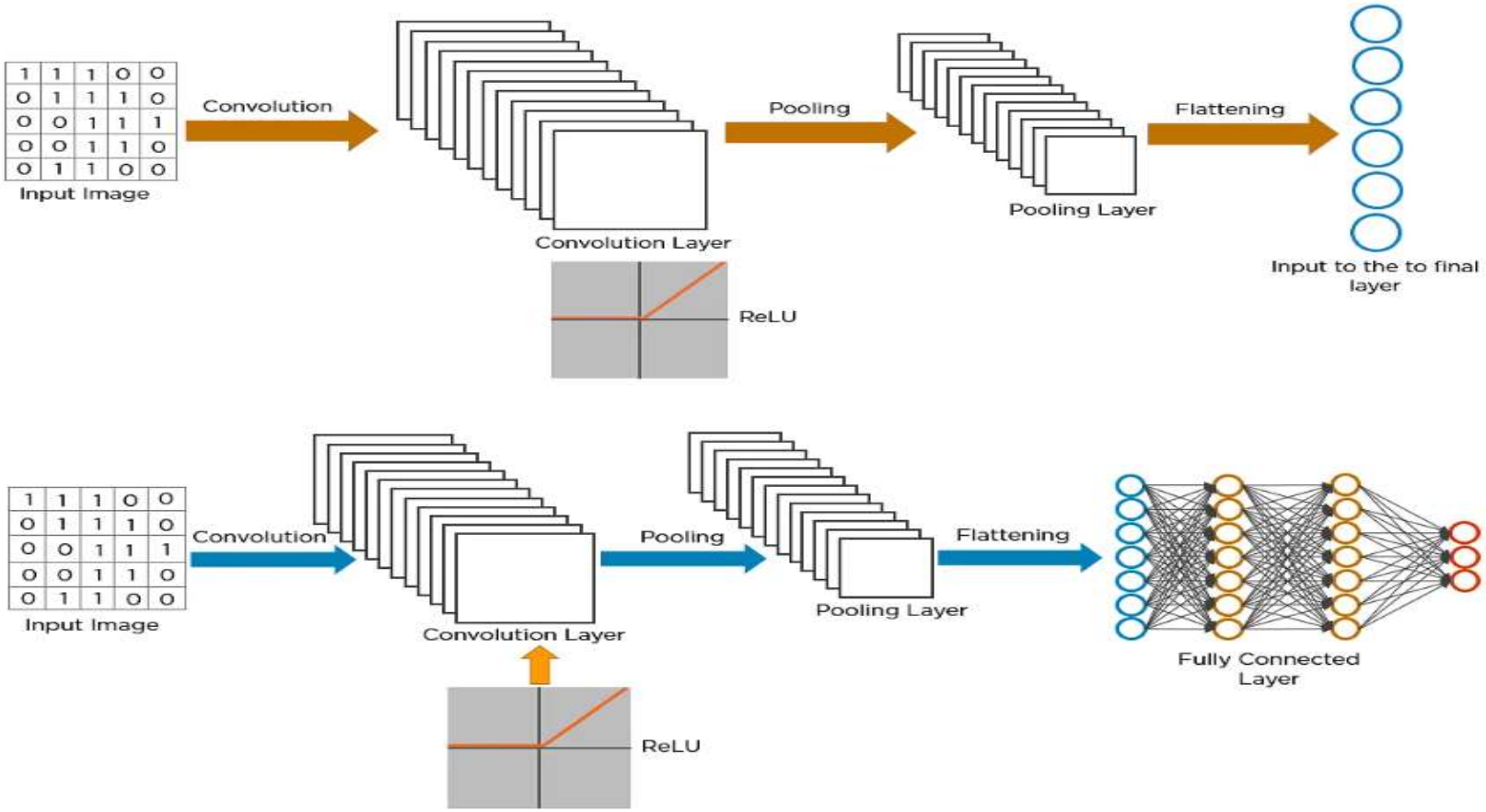
Flattening

- The next step in the process is called flattening.
- Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector.

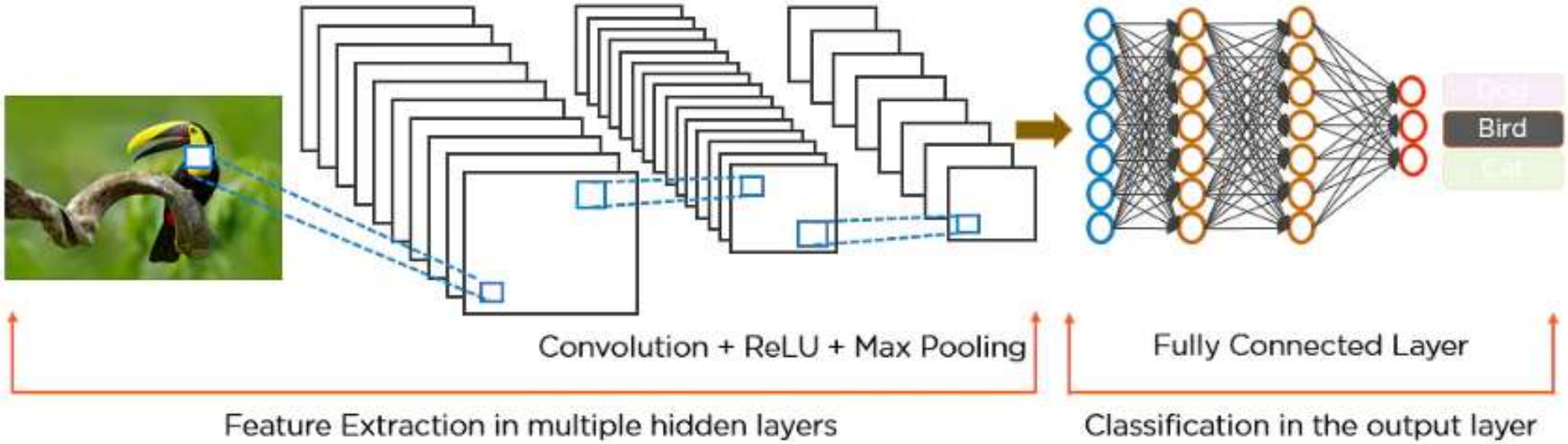


Fully Connected Layer

- The flattened matrix is fed as input to the fully connected layer to classify the image.



Fully Connected Layer



Different CNN Architectures

LeNet-5 (1998)

- **Key Idea:** One of the earliest CNNs, designed for digit recognition. Simple architecture with convolution + pooling + fully connected layers.
- **Use Case:**
  - Handwritten digit recognition (e.g., MNIST dataset)
  - Simple image classification tasks

Different CNN Architectures

AlexNet (2012)

- **Key Idea:** Deep CNN with 8 layers, ReLU activation, dropout for regularization, trained on GPUs. Sparked the deep learning revolution in computer vision.
- **Use Case:**
  - Large-scale image classification (ImageNet)
  - Feature extraction for transfer learning

Different CNN Architectures

VGGNet (2014)

- **Key Idea:** Uses small 3×3 filters stacked in depth, leading to very deep networks (VGG-16, VGG-19).
- **Use Case:**
  - Image classification (benchmarking standard)
  - Transfer learning for tasks like medical imaging, face recognition

Different CNN Architectures

GoogLeNet / Inception (2014)

- **Key Idea:** Inception modules (parallel convolution filters of different sizes + pooling) to capture multi-scale features efficiently.
- **Use Case:**
  - Image recognition with fewer parameters
  - Applications where computational efficiency matters (e.g., mobile vision tasks)

Different CNN Architectures

ResNet (2015)

- **Key Idea:** Introduced **residual connections** (skip connections) to train very deep networks (e.g., ResNet-50, ResNet-101). Solves vanishing gradient problem.
- **Use Case:**
  - Large-scale image recognition (ImageNet)
  - Medical imaging (X-rays, CT scans)
  - Feature extraction in many AI systems

Different CNN Architectures

DenseNet (2017)

- **Key Idea:** Each layer connects to every other layer (dense connectivity). Improves gradient flow and reduces parameters.
- **Use Case:**
  - Image classification
  - Object recognition in cluttered backgrounds
  - Medical diagnosis (e.g., lesion detection)

Different CNN Architectures

MobileNet (2017)

- **Key Idea:** Lightweight CNN using **depthwise separable convolutions** for mobile/embedded devices.
- **Use Case:**
  - Mobile vision (face unlock, AR apps)
  - Edge AI (IoT cameras, robotics)

Different CNN Architectures

EfficientNet (2019)

- **Key Idea:** Compound scaling (depth, width, resolution) to optimize accuracy vs. efficiency trade-off.
- **Use Case:**
  - High-accuracy image classification with fewer resources
  - Medical imaging, security surveillance
  - Deployment in production environments

Different CNN Architectures

Region-based CNNs (R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN)

- **Key Idea:** Extend CNNs to **object detection and segmentation** by predicting bounding boxes and class labels.
- **Use Case:**
  - Self-driving cars (detect pedestrians, traffic signs)
  - Medical image segmentation (tumors, organs)
  - Video surveillance

Different CNN Architectures

YOLO (You Only Look Once)

- **Key Idea:** Real-time object detection with a single CNN predicting bounding boxes + classes in one pass.
- **Use Case:**
  - Real-time detection (autonomous driving, drones, robotics)
  - Retail (product detection in stores)
  - Sports analytics (tracking players/ball)

Different CNN Architectures

U-Net (2015)

- **Key Idea:** Encoder–decoder CNN with skip connections, specialized for pixel-level segmentation.
- **Use Case:**
  - Biomedical image segmentation (cells, organs, tumors)
  - Satellite image segmentation (roads, land cover)